Open-Source Software Audit Report

A Capstone Project for OSS NGMC Course

Student Name:	Prince Bhandari
Registration Number:	24BCY10302
Software Chosen:	Git
Environment:	Oracle VirtualBox KaliLinux
Category:	Version Control
License:	GPL v2
Course:	Open-Source Software

INTRODUCTION

Open-source software is software that anyone can freely use, modify and share. It forms the backbone of modern computing, powering operating systems, development tools, and the web.

This report audits Git — the world's most widely used distributed version control system. Created by Linus Torvalds in 2005, Git revolutionized how developers collaborate on code. This audit examines its origin, licensing, ethical considerations, Linux footprint, and open-source ecosystem.

PART A — ORIGIN STORY AND PHILOSOPHY

Git was created by Linus Torvalds in April 2005 after a licensing dispute ended free use of BitKeeper for Linux kernel development. Torvalds needed a fast, distributed, and fully free version control system — so he built one himself in just a few weeks.

A.1 PROBLEM SOLUTION

Before Git, developers used centralised tools like CVS and Subversion (SVN), where a single server held the full history. Git changed everything: every developer has a complete local copy of the repository, enabling offline work, fast branching, and merging at scale.

Problems Before Git

- Single point of failure: All code history was stored on a central server. If it went down, work stopped.
- Slow operations: Branching and merging were difficult and time-consuming.
- Limited offline work: Developers needed constant access to the central server.
- Poor scalability: Large projects like the Linux kernel struggled with performance.

- Trust issues: Centralized control meant less flexibility for distributed teams.

Torvalds chose to release Git as open source because the Linux project itself was open source. Collaboration required transparency and he wanted to avoid future dependency on proprietary tools. Also open development leads to faster innovation and trust.

Git quickly became the foundation for platforms like GitHub (2008), GitLab, and Bitbucket, transforming how the entire software industry collaborates.

A.2 LICENSE ANALYSIS

Git is licensed under the GNU General Public License v2 (GPLv2) — a copyleft license. This means anyone can use, study, modify, and distribute Git freely. However, any modified version that is distributed must also be released as open source under GPLv2.

Unlike permissive licenses such as MIT or Apache, GPLv2 prevents modified versions from becoming proprietary. This copyleft design ensures Git remains free and open forever, even if a company builds on top of it. Git's GPLv2 license grants all four freedoms.

The four core freedoms granted are: the right to use, study, modify, and share the software.

A.3 ETHICS OF OPEN SOURCE

Open source promotes transparency, collaboration, and shared progress. However, ethical questions arise:

- Companies sometimes build profitable products on open-source work without contributing back. Permissive licenses allow this; GPLv2 used by Git prevents it.

- Volunteer maintainer burnout is a growing concern — critical software relied on by millions is often maintained by a handful of unpaid contributors.

- Open source is not just about free software — it is a philosophy of shared knowledge, collective growth, and community responsibility.

PART B — LINUX FOOTPRINT

The analysis was carried out on Kali Linux. Git is installed system - wide and runs only when called from the terminal — it has no background daemon or service.

Key locations:

- Executable: /usr/bin/git
- User config: ~/.gitconfig
- Per-repo config: .git/config (inside each project)
- Installation: sudo apt install git

User-level settings such as name and email are stored in ~/.gitconfig. Updates are handled via the apt package manager.

PART C — FOSS ECOSYSTEM

Git sits at the centre of a vast open-source ecosystem:

- Hosting Platforms: GitHub, GitLab, Bitbucket
- GUI Tools: lazygit, gitk, Sourcetree
- IDE Integrations: VS Code, Vim, JetBrains IDEs
- CI/CD: GitHub Actions, Jenkins, CircleCI — all triggered by Git events
- Package Managers: npm, pip, cargo all resolve packages from Git repos

Git is actively maintained by the open-source community at kernel.org and is a foundational dependency for virtually every modern software project.

PART D — COMPARISON: GIT VS SVN

Feature	Git	SVN / Subversion
Model	Distributed	Centralised
License	GPLv2	Apache 2.0
Cost	Free and open source	Free and open source
Architecture	Distributed Version Control System (DVCS)	Centralized Version Control System (CVCS)
Speed	Very fast (local ops)	Slower (needs server)
Community vs Corporate Control	Community-driven; widely used across individuals, startups and enterprises	More enterprise-oriented; often used in corporate environments with controlled access
Branching	Lightweight & instant	Heavy and slow
Offline Work	Full functionality	Very limited
Support & Reliability	Highly reliable due to distributed nature	Reliable but depends on central server
Learning Curve	Steeper initially	Easier for beginners

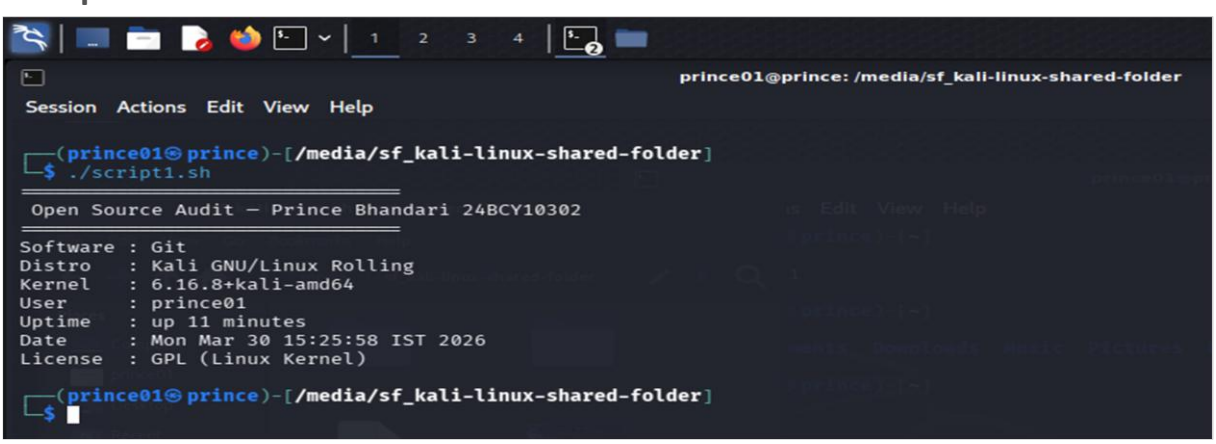
Git's distributed model and GPLv2 license make it more resilient and community-friendly than SVN. While SVN is easier for beginners, Git's performance, flexibility, and ecosystem make it the clear choice for modern software development.

SCRIPT 1 — SYSTEM IDENTITY REPORT

Code

```
#!/bin/bash
# Script 1: System Identity Report

STUDENT_NAME="Prince Bhandari
24BCY10302" SOFTWARE_CHOICE="Git"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)

DISTRO=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')

echo "================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "================================"
echo "Software : $SOFTWARE_CHOICE"
echo "Distro   : $DISTRO"
echo "Kernel   : $KERNEL"
echo "User     : $USER_NAME"
echo "Uptime   : $UPTIME"
echo "Date     : $DATE"
echo "License  : GPL (Linux Kernel)"
```

Output



Explanation

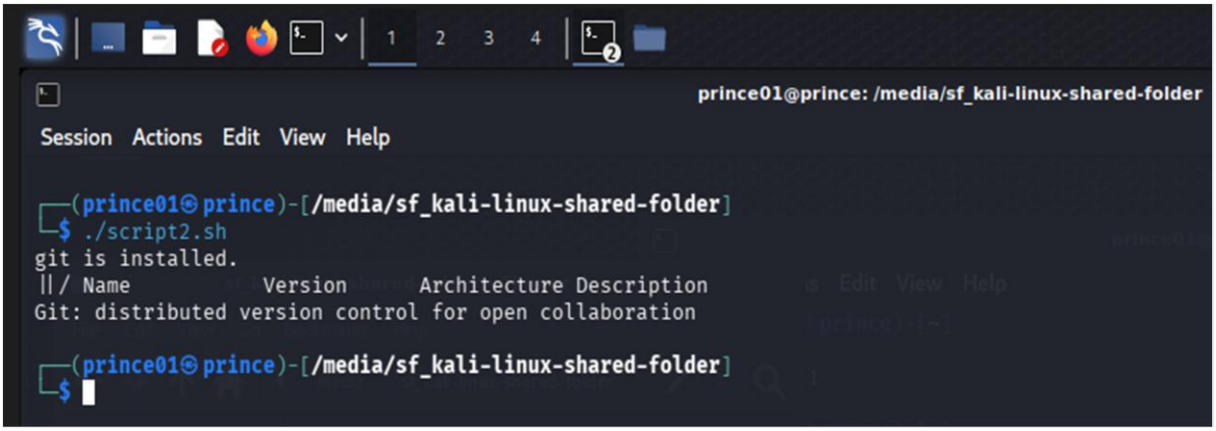
Displays key system information: distro, kernel version, current user, uptime, and date. Uses variables and command substitution $() to fetch live system data, and echo to format the output.

SCRIPT 2 — FOSS PACKAGE INSPECTOR

Code

```
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE="git"

if dpkg -l | grep -q "^ii  $PACKAGE"; then
    echo "$PACKAGE is installed."
    dpkg -l $PACKAGE | grep -E 'Version'
else
    echo "$PACKAGE is NOT installed."
fi

case $PACKAGE in
    git) echo "Git: distributed version control for open collaboration" ;;
    apache2) echo "Apache: web server powering internet" ;;
    mysql) echo "MySQL: open database system" ;;
    vlc) echo "VLC: open media player" ;;
esac
```

Output



Explanation

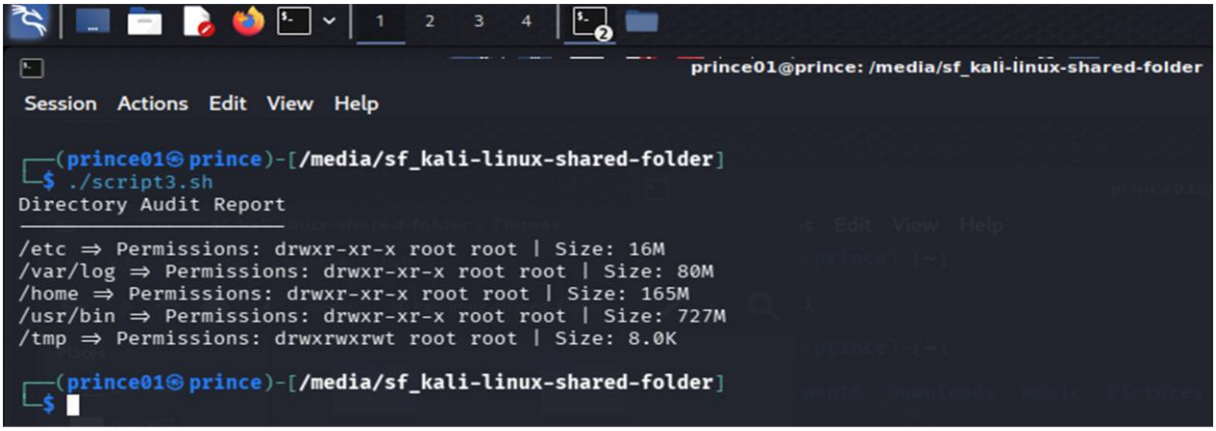
Checks if Git is installed using dpkg and displays its version. A case statement provides short descriptions for common FOSS packages. Demonstrates if/else, package querying, and conditional branching.

SCRIPT 3 — DISK AND PERMISSION AUDITOR

Code

```
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "-------------------------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist"
    fi
done
```

Output



Explanation

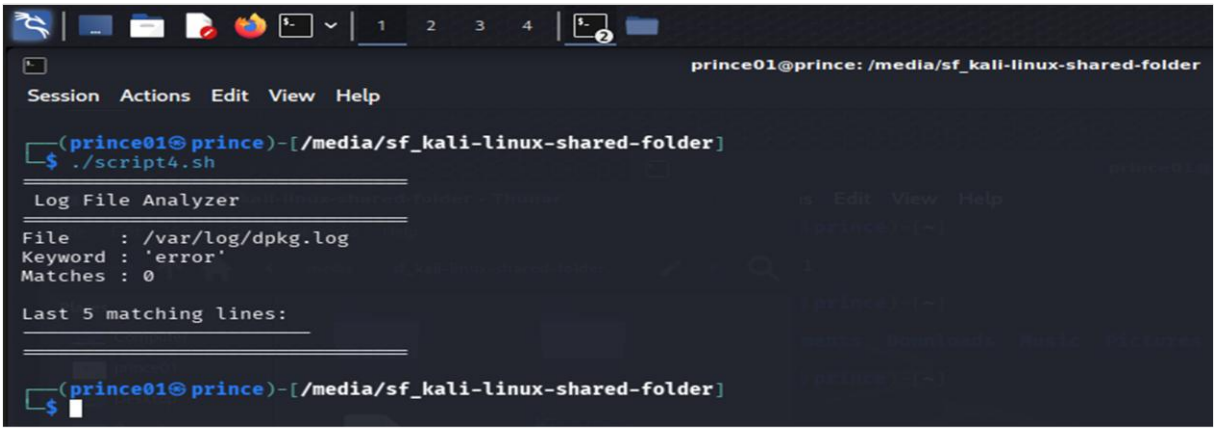
Loops through key system directories using a Bash array and for loop. Displays permissions with ls -ld and disk usage with du -sh. Demonstrates arrays, loops, and command output parsing with awk.

SCRIPT 4 — LOG FILE ANALYZER

Code

```
#!/bin/bash
# Script 4: Log File Analyzer
if [ -n "$1" ]; then
    LOGFILE="$1"
elif [ -f "/var/log/kern.log" ]; then
    LOGFILE="/var/log/kern.log"
elif [ -f "/var/log/auth.log" ]; then
    LOGFILE="/var/log/auth.log"
elif [ -f "/var/log/dpkg.log" ]; then
    LOGFILE="/var/log/dpkg.log"
else
    echo "No log file found."; exit 1
fi

KEYWORD="${2:-error}"
if [ ! -r "$LOGFILE" ]; then
    echo "Cannot read $LOGFILE. Try sudo."; exit 1
fi
COUNT=$(grep -ic "$KEYWORD" "$LOGFILE" 2>/dev/null)
echo "File    : $LOGFILE"
echo "Keyword : '$KEYWORD'"
echo "Matches : $COUNT"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Output



Explanation

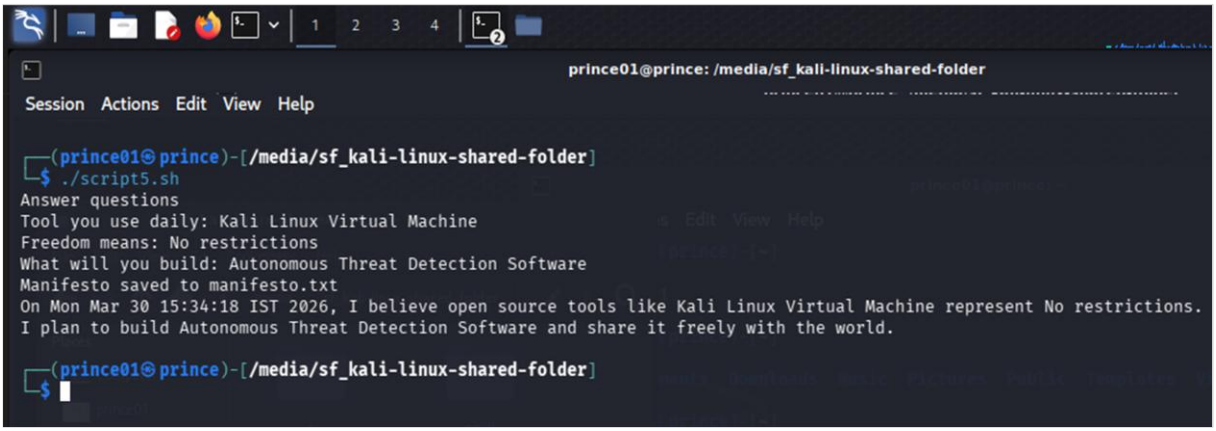
Analyzes a log file for keyword matches. Auto-detects available log files on the system. Uses grep -ic for case-insensitive counting and tail to show the last 5 matching lines. Includes file existence and readability checks.

SCRIPT 5 — OPEN SOURCE MANIFESTO GENERATOR

Code

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
echo "Answer questions"

read -p "Tool you use daily: " TOOL
read -p "Freedom means: " FREEDOM
read -p "What will you build: " BUILD

DATE=$(date)
OUTPUT="manifesto.txt"

echo "On $DATE, I believe open source tools like $TOOL represent $FREEDOM." > $OUTPUT
echo "I plan to build $BUILD and share it freely with the world." >> $OUTPUT

echo "Manifesto saved to $OUTPUT"
cat $OUTPUT
```

Output



Explanation

Takes three interactive inputs using read -p and builds a personalised open-source manifesto, saving it to a file using > and >> redirection. Demonstrates user input, string interpolation, and file writing.

CONCLUSION

This audit of Git reveals how a tool built out of necessity — in just a few weeks — became foundational infrastructure for the entire software industry.

Key takeaways:

- GPLv2 licensing ensures Git remains free and open-source forever, preventing it from being made proprietary.

- Its distributed model solved real problems that centralised VCS tools like SVN could not handle at scale.

- The FOSS ecosystem around Git — GitHub, CI/CD pipelines, and package managers— multiplies its value enormously.

- Open source is about more than free software. It is about transparency, collaboration, and shared responsibility among developers worldwide.